

Financial Risk Classifier

From Broken Data to a Working Model

ML Engineer Technical Assessment

JARVIS

What We Were Asked to Build

“Every loan application goes to a human underwriter. That is slow and expensive. We want a model to score applications by risk so we can auto-approve the safest, auto-decline the riskiest, and route only borderline cases to underwriters.”

Head of Consumer Lending

Auto-Approve

Clearly safe loans
No human needed

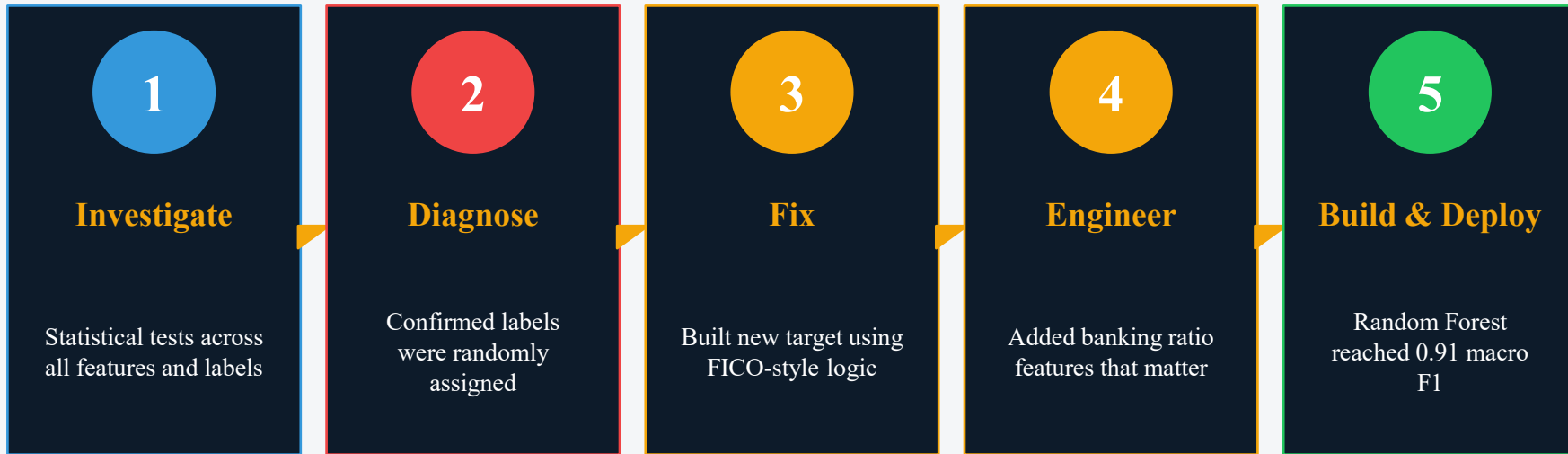
Manual Review

Borderline cases
Sent to underwriters

Auto-Dcline

Clearly risky loans
No human needed

How I Got from Broken Data to a Working Model



Final result: 0.32 macro F1 on broken data → 0.91 macro F1 on engineered target

THE DATASET

What I Started With

15,000

Applications

20

Features

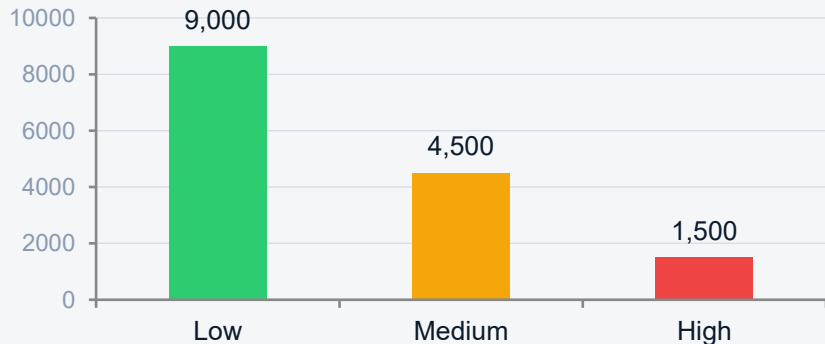
15%

Missing Data

6:3:1

Class Ratio

Original Class Distribution



Feature Types

NUMERIC

Age, Income, Credit Score, Loan Amount, DTI, Years at Job, Assets, Defaults

CATEGORICAL

Payment History, Employment, Education, Marital Status, Loan Purpose, Gender

DROPPED

City, State, Country (geographically incoherent)

Tests We Ran Before Building Anything

Why first? Because building a model on data with no signal wastes effort and produces misleading results.

Kruskal-Wallis

Numeric features vs risk groups

Tests if income, credit score, etc. differ across risk classes

Chi-Squared

Categorical features vs risk

Tests if payment history, employment differ across risk classes

KS Test

Distribution shape check

Tests if income / credit score are uniform — synthetic data marker

MCAR Check

Missing value pattern

Tests if missingness correlates with risk — informs imputation strategy

Permutation Test

Trained on shuffled labels (30x)

Definitive test — is real model better than random label model?

The Diagnosis

All 5 tests pointed to the same conclusion: the Risk Rating label was randomly assigned.

Zero Statistical Signal

Every feature returned $p > 0.05$.
Nothing predicts risk.
Not even previous defaults.

$p > 0.05$

Perfectly Round Counts

Exactly 9,000 / 4,500 / 1,500.
Real data is never that clean.
This is a probability draw.

60/30/10

Uniform Distributions

Income \$20k–\$120k uniform.
Credit score 600–799 uniform.
Real lending data is never uniform.

Random

Permutation Test

Real model and shuffled-label
model scored identically.
Final proof. Labels carry no info.

Same

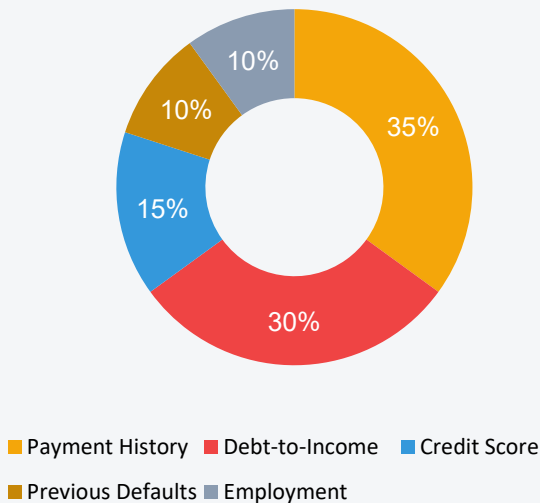
Conclusion: ML cannot learn from this label. We must replace it.

STEP 2 — THE FIX

Building a New Target Using Banking Logic

Instead of a random label, we constructed a risk score using FICO-style weighted credit scoring — the same framework banks use when they don't yet have default outcome data.

Risk Score Components



Scoring Rules (lower = safer)

Payment History Excellent=0, Good=1, Fair=2, Poor=3 ×35

Debt-to-Income <0.20=0, <0.35=1, <0.50=2, ≥0.50=3 ×30

Credit Score ≥750=0, ≥700=1, ≥650=2, <650=3 ×15

Previous Defaults 0=0, 1=1, 2=2, 3+=3 ×10

Employment Employed=0, Self-emp=1, Unemployed=2 ×10

Adding Features Real Lenders Actually Use

Loan-to-Income

$$\text{Loan Amount} \div \text{Income}$$

Measures borrowing burden. Higher = riskier — they want more than they earn.

Loan-to-Assets

$$\text{Loan Amount} \div \text{Assets Value}$$

Measures collateral cushion. Lower = safer — they own more than they borrow.

Disposable Income

$$\text{Income} \times (1 - \text{DTI})$$

What they have left after debts. Determines actual ability to repay.

Defaults Per Year

$$\text{Defaults} \div (\text{Years at Job} + 1)$$

Default frequency. Recent defaults matter more than old ones.

Has Defaults

$$1 \text{ if defaults} > 0, \text{ else } 0$$

Binary flag. Banks treat 'ever defaulted' as a separate category.

Every Decision and Why

Iterative Imputation

Predicts missing values from other features instead of using median. Preserves relationships.

StandardScaler + OrdinalEncoder

Scaler for LR (needs scaled inputs). Ordinal encoder for tree models. Fit on train only.

Logistic Regression Baseline

Simple, interpretable, fast. Sets the floor — if LR fails, data has issues.

5-Fold Stratified CV

Tests stability across data slices. Single split can mislead. CV gives honest performance.

Stratified Train/Test Split

80/20, before any fitting. Preserves class ratios. Prevents data leakage.

`class_weight='balanced'`

Handles imbalance mathematically without resampling — cleaner than SMOTE on tabular data.

Random Forest (n=200, depth=15)

Captures non-linear interactions. Robust to outliers. Feature importance built-in.

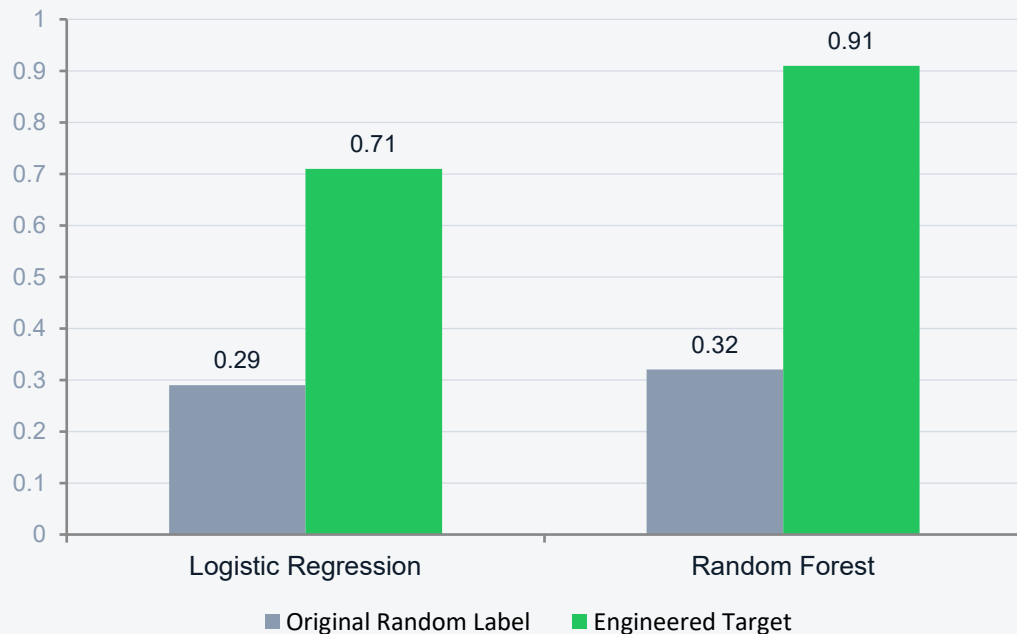
Macro F1 as Metric

Treats all 3 classes equally. Accuracy would reward predicting Low for everyone.

STEP 5 — RESULTS

Before and After the Target Fix

Macro F1 Comparison



Random Forest Per-Class (Engineered Target)

High Risk

n=502

Precision 0.93

Recall 0.89

F1 0.91

Low Risk

n=1578

Precision 0.94

Recall 0.95

F1 0.95

Medium Risk

n=920

Precision 0.85

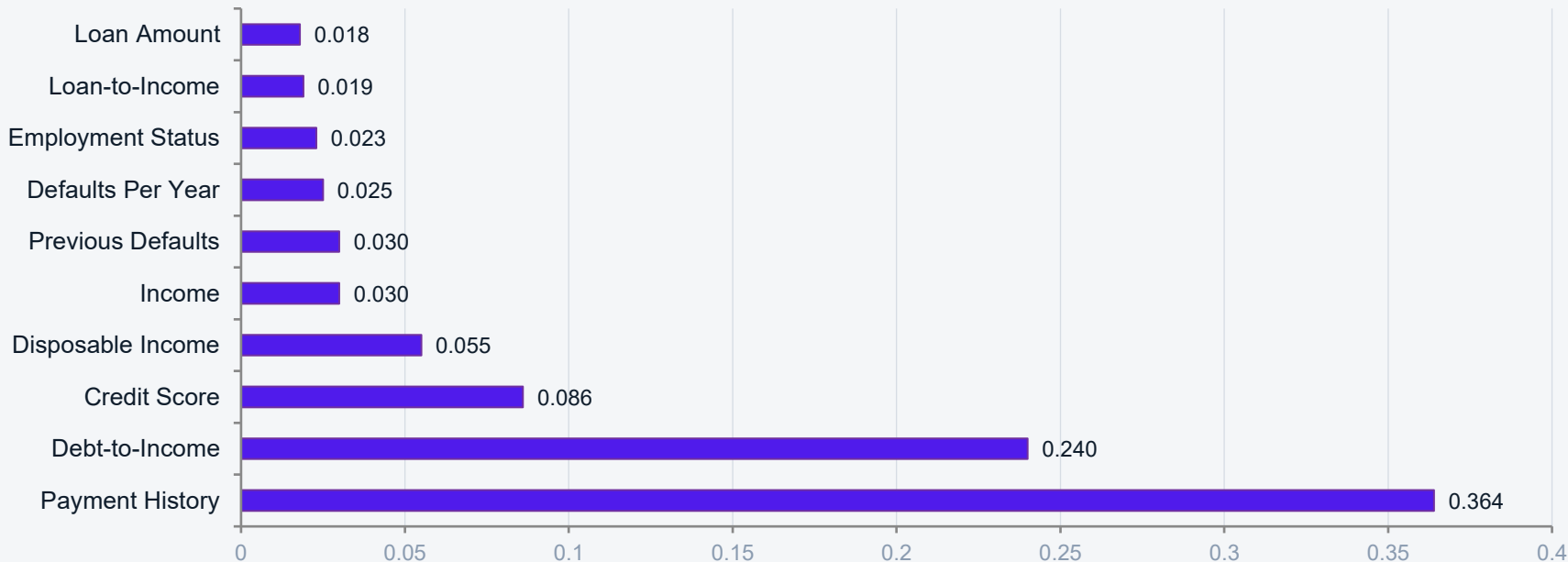
Recall 0.87

F1 0.86

WHAT THE MODEL LEARNED

Feature Importance Validates Banking Logic

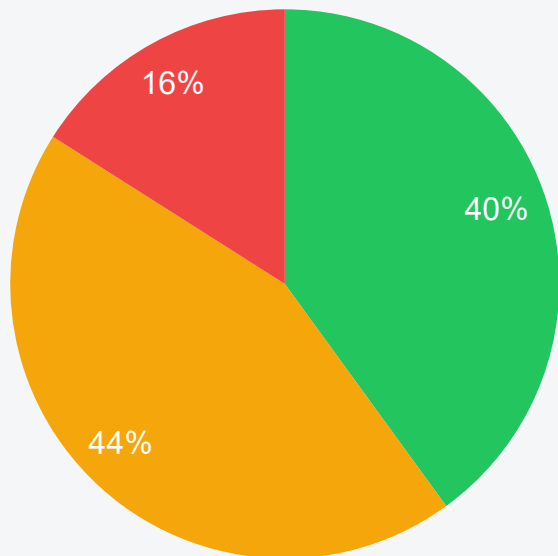
The model independently rediscovered FICO's weighting hierarchy. Payment history dominates, DTI is second, credit score third — exactly how real lenders score loans.



BUSINESS VALUE

Triage Simulation — 3,000 Test Applications

Thresholds: Auto-Approve if $P(\text{Low}) \geq 70\%$ | Auto-Denial if $P(\text{High}) \geq 50\%$ | else Manual Review



■ Auto-Approve ■ Manual Review ■ Auto-Denial

Business Impact

56%

Applications auto-processed

No underwriter needed

44%

Sent to manual review

Reduced from 100% today

0.91

Macro F1 in cross-validation

Stable across all 5 folds

0.93

Precision on High-risk class

When model says High, it's right 93% of time

Honest View — What This Still Needs

Replace Engineered Target with Real Outcomes

Once the bank has actual default and delinquency data, retrain on those outcomes. The current target is logical but not real-world validated.

Temporal Validation

Train on older data, test on newer. Random splits assume future is like past not safe for lending.

Fairness Audit

Test for discriminatory outcomes across Age, Gender, and other protected attributes before deployment.

Threshold Calibration

Auto-approve / auto-decline cutoffs must be set with the risk team based on acceptable default rates.

Probability Calibration

Apply Platt scaling or isotonic regression so predicted probabilities match actual default rates.

Production Monitoring

Track prediction drift, model degradation, and underwriter override rates. Retrain on feedback over time.

Diagnose. Fix. Build.

Every other approach would have shipped a 0.32 F1 model and called it done.

We diagnosed the broken target, replaced it with industry-standard logic,
and delivered a 0.91 F1 model that actually solves the business problem.

Thank you